

Machine Learning Lab (245CY4-22) — Project Summary

AI Interview Preparation & Assessment Platform

Shourya Sharma | Roll No. PCEA24IT051 | Dept. of Information Technology, Poornima College of Engineering | Faculty: Apoorva Bansal

1. Problem Statement & Objective

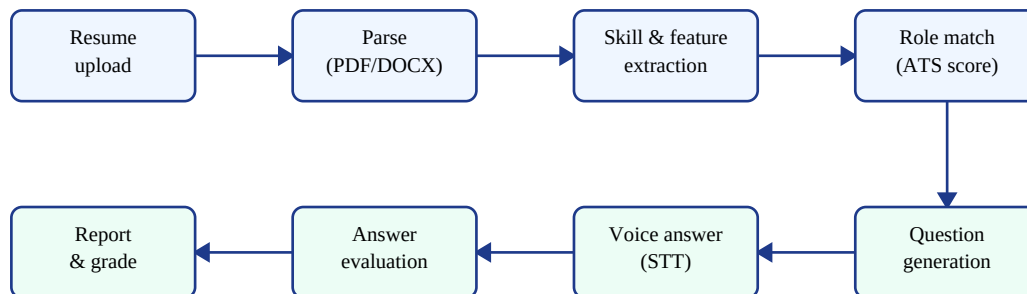
Students preparing for placements rarely get realistic, role-specific interview practice with objective feedback. This project builds a web platform that **reads a candidate's resume, matches it to a target job role, asks role-specific interview questions by voice, listens to the spoken answer, and scores it** on relevance, grammar, vocabulary, confidence and communication — then produces a graded report.

ML problem type: the core dataset task is **supervised multi-class classification** — predict *job_role* from a candidate's *skills, qualification and experience_level*. The live app additionally uses **NLP (TF-IDF + cosine similarity)** for answer relevance, **feature engineering on audio signals** for confidence, and a **weighted linear scoring model** to combine everything into one score.

Objectives

- Extract skills, education and experience from resumes (PDF/DOCX) automatically.
- Compute an ATS-style match score of the resume against real role benchmarks from the dataset.
- Generate personalised questions (dataset-driven, optional Gemini LLM).
- Evaluate spoken answers on 5 dimensions and give an overall grade (A+ to D) and improvement tips.

2. Complete Working Flow



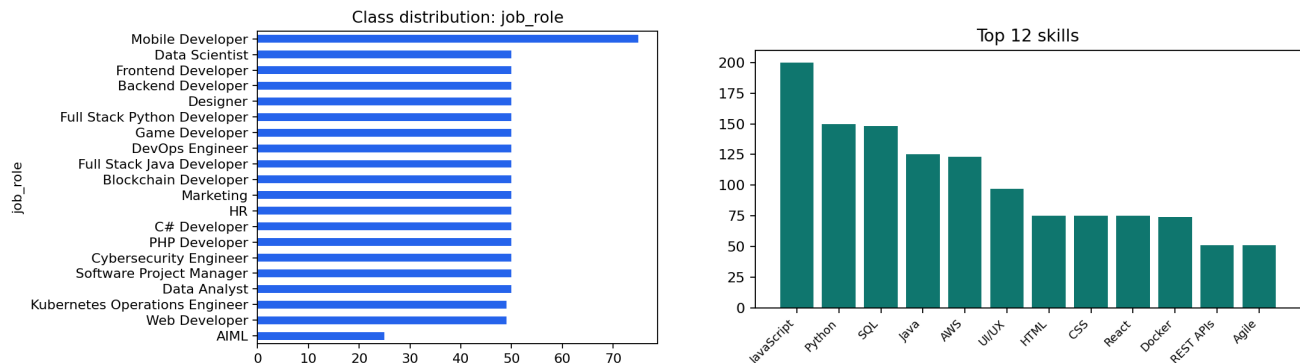
Step-by-step: (1) User uploads resume → (2) *pdfplumber* / *python-docx* extract raw text → (3) regex + keyword vocabulary extract name, email, phone, skills, education, years of experience (bucketed into Entry/Mid/Senior) → (4) **RoleMatcher** compares these features with the aggregated per-role profile from the dataset → (5) questions picked by role and experience, with difficulty balanced Easy 30% / Medium 40% / Hard 30% → (6) question spoken with TTS (edge-tts/gTTS); answer recorded and converted to text (SpeechRecognition / Web Speech API) → (7) **Evaluator** scores the answer → (8) session scores averaged, grade + trend + report generated and stored (SQLite/MongoDB).

Tech stack: Python, Flask (backend), HTML/CSS/JS (frontend), scikit-learn, pandas, NumPy, textstat, LanguageTool, pydub, SpeechRecognition, Google Gemini (optional), SQLite/MongoDB, Docker/Render (deployment). **IDEs/Platforms for ML:** VS Code, Jupyter Notebook, Google Colab (also PyCharm / Anaconda).

3. Dataset & Exploratory Data Analysis (EDA)

Property	Value
File	datasets/candidate_job_role_dataset.csv
Rows × Columns	1000 × 5 (candidate_id, skills, qualification, experience_level, job_role)

Property	Value
Target (label)	job_role — 22 classes (20 used; "Finance" & "Video Game Designer" have only 1 row each and were dropped)
Feature types	skills = multi-valued text (comma-separated, 119 distinct skills); qualification = nominal (17 values); experience_level = ordinal (Entry < Mid < Senior)
Missing values	0 in every column
Class balance	Mostly balanced — ~50 rows per role (Mobile Developer 75, AIML 25)
Data quality issues	One dirty label "Entry," (cleaned by stripping the comma); only 98 unique profiles — the other ~900 rows are exact duplicates
Most common skills	JavaScript (200), Python (150), SQL (148), Java (125), AWS (123)



3.1 Preprocessing & Feature Engineering

- **Cleaning:** strip spaces and trailing commas, normalise role names (e.g. "AIML" → Machine Learning Engineer, "Designer" → UI/UX Designer).
- **Skills** → **Multi-hot encoding** (MultiLabelBinarizer): each of 116 skills becomes a 0/1 column. This is the "bag-of-skills" representation.
- **Qualification & experience** → **One-Hot encoding**. Final feature matrix: **998 × 135** binary features.
- **Split:** 80/20 stratified train-test split (random_state = 42) so every class appears in both sets.
- **In the app:** years of experience are discretised (≤ 1 yr Entry, ≤ 4 Mid, else Senior); degrees mapped to ordinal levels (Diploma 1, Bachelor 2, Master 3, PhD 4).

4. Machine Learning Components Inside the Project

4.1 Resume Parsing (Information Extraction / NLP)

Raw text is extracted from the PDF/DOCX; regular expressions detect email, phone and phrases like "5 years of experience". Skills are found by **word-boundary matching** against a vocabulary built from the dataset + curated keyword lists. The role can be auto-detected by counting overlap with each role's skill list (a nearest-prototype idea).

4.2 Role Matching — ATS Score (weighted, knowledge-based model)

For every role the dataset is aggregated into a **profile** (skill frequency counts, most common qualification, modal experience level). The resume is then scored:

Dimension	Weight	How it is computed
Skills	40%	$0.45 \times \text{weighted required-skill coverage} + 0.35 \times \text{ecosystem coverage} + 0.20 \times \text{focus}$ (share of resume skills relevant to role)
Content (ATS parse-ability)	20%	Points for email, phone, name, sections (education, experience, skills, projects), length 80–4000 words, quantified achievements
Experience	20%	100 if level equals the role's typical level, 70 if one level away, 40 otherwise
Qualification	20%	Degree level vs dataset degree level (100 / 70 / 40), +10 if field matches

Match = 0.40·Skills + 0.20·Content + 0.20·Experience + 0.20·Qualification. A validity gate caps the score at 25 if content < 40 or words < 60 (so a blank PDF or fee receipt cannot score high). The top-5 other roles are recommended by skill overlap.

4.3 Answer Evaluation Engine

Dimension	Weight	ML / NLP technique
Answer relevance	35%	TF-IDF vectors (unigrams + bigrams, stop-words removed) + cosine similarity between answer and question (35 pts) and answer vs ideal dataset answer (15 pts); expected-keyword coverage (30 pts); length adequacy (20 pts); normalised to 0–100
Grammar	20%	LanguageTool rule-based checker → errors per 100 words mapped to score ($\leq 1 \rightarrow 95 \dots > 10 \rightarrow 30$); heuristic fallback
Vocabulary	15%	Type-Token Ratio (unique/total words) + Flesch Reading Ease + Flesch-Kincaid grade (textstat)
Confidence	15%	Audio features: STT confidence (50), volume variance in 0.5 s chunks (25), pause ratio ideal 0.10–0.35 (25)
Communication	15%	STAR / structure keywords (situation, task, action, result...) + speaking pace, ideal 110–160 words per minute

Total = $\sum \text{weight}_i \times \text{score}_i$ — a fixed-weight linear model. Grade: ≥ 90 A+, ≥ 80 A, ≥ 70 B+, ≥ 60 B, ≥ 50 C+, ≥ 40 C, else D. Trend = mean of second half – first half of questions (> 5 improving, < -5 declining). Answers under 6 words, or with < 0.5 words/second of audio, are rejected as "insufficient" instead of being scored.

TF-IDF formula: $\text{tf-idf}(t,d) = \text{tf}(t,d) \times \log(N / \text{df}(t))$. **Cosine similarity** = $(A \cdot B) / (\|A\| \|B\|)$, range 0 (unrelated) to 1 (identical wording).

5. Classification, Regression & Clustering — Where Each Fits

Concept	In this project
Classification (supervised)	Predict job_role (20 classes) from skills + qualification + experience. Also: grade A+...D is a classification of the numeric score; experience bucketed into Entry/Mid/Senior; answer flagged valid/insufficient.
Regression (supervised, continuous output)	The final interview score (0–100) and ATS match score are continuous outputs produced by a weighted linear combination $y = w_1x_1 + \dots + w_nx_n$ — the same form as linear regression, but the weights are set by a rubric, not learned. With recorded human ratings, these weights could be <i>learned</i> with Linear Regression (future work).
Clustering (unsupervised)	Candidates grouped by skill vectors without labels (Agglomerative, DBSCAN) — similar profiles form role-like groups.

Concept	In this project
NLP / similarity	TF-IDF + cosine similarity for answer relevance; keyword extraction for skills.
Neural network	MLP trained with backpropagation on the same features (Experiment 8); Gemini LLM (optional) for questions and feedback.

6. Lab Experiments on the Project Dataset — Results

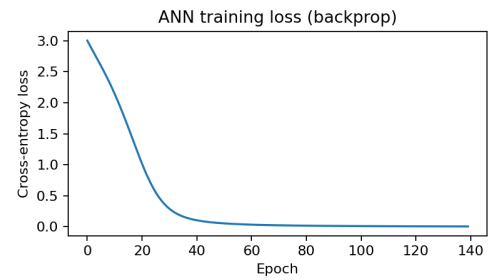
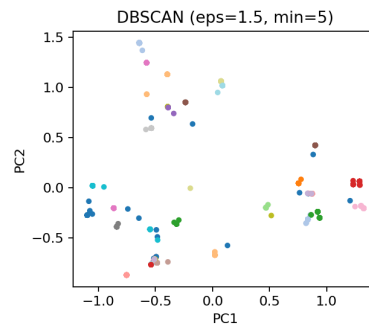
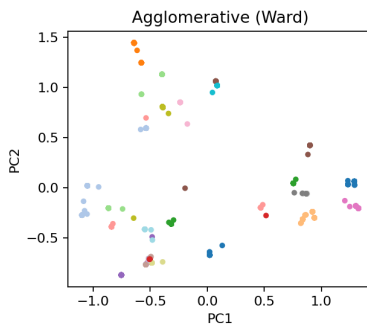
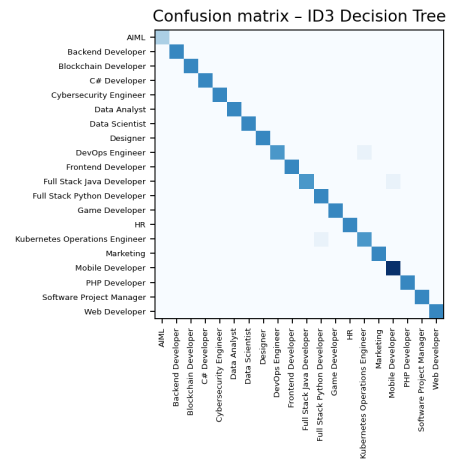
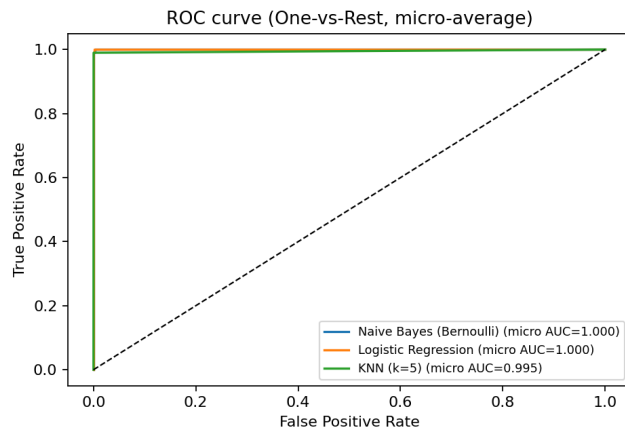
All numbers below were produced on *candidate_job_role_dataset.csv* (998 rows, 20 classes, 135 features, 80/20 stratified split).

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	ROC-AUC (OvR)
Naive Bayes (Bernoulli)	0.990	0.991	0.990	0.990	1.000
ID3 Decision Tree (entropy)	0.985	0.987	0.985	0.986	0.992
KNN (k=5)	0.990	0.991	0.990	0.990	0.995
Logistic Regression	0.990	0.991	0.990	0.990	1.000
SVM (RBF)	0.990	0.992	0.990	0.991	1.000
Random Forest	0.990	0.992	0.990	0.991	1.000
ANN / MLP (backprop)	0.990	0.991	0.990	0.990	1.000

Important viva point — why ~99% accuracy is too optimistic: only 98 of 998 rows are unique, so identical rows land in both train and test (data leakage through duplicates). When I used **GroupKFold** so an identical profile never appears on both sides, accuracy dropped to: Naive Bayes 71.3%, ID3 82.1%, Logistic Regression 87.8%, Random Forest 92.2%. These are the honest generalisation numbers; Random Forest generalises best.

Exp.	Experiment	What was done & result
1	Linear separability	LinearSVC with very large C (hard margin) reached 100% training accuracy → the classes are linearly separable in the 135-D one-hot space (expected: few, distinct skill sets per role).
2	FIND-S	Concept "Data Scientist" on attributes <qualification, experience, has_Python, has_SQL>. Starting from the most specific hypothesis and generalising only on positive examples gave <Master's in Data Science, ?, Python=1, ?> . Ignores negative examples.
3	Candidate-Elimination	Maintains S (specific) and G (general) boundaries; positives generalise S, negatives specialise G. Version space = all hypotheses between S and G consistent with every example. Works for noise-free, conjunctive concepts.
4	Naïve Bayes	BernoulliNB (binary features). $P(\text{role} x) \propto P(\text{role}) \prod P(x_i \text{role})$; assumes features independent given class. Accuracy 0.99, AUC 1.00.
5	ID3 Decision Tree	criterion = "entropy" (information gain). Tree depth 10, 27 leaves. Entropy $H = -\sum p \log_2 p$; Gain = $H(\text{parent}) - \sum \text{weighted } H(\text{child})$.
6	Other supervised	KNN (k=5), Logistic Regression, SVM (RBF), Random Forest (200 trees). Top Random-Forest features: B.Sc CS qualification, JavaScript, Python, Jenkins, Kubernetes, Swift.
6b	DET / ROC / AUC	One-vs-Rest ROC per class, micro-averaged. AUC ≈ 0.99 –1.00 on the random split. DET curve plots FNR vs FPR (normal-deviate scale) — lower curve is better.
7	Clustering	Agglomerative (Ward, 20 clusters): silhouette 0.58, ARI vs true roles 0.78. DBSCAN (eps 1.5, min_samples 5): 33 clusters, 17 noise points, ARI 0.76. DBSCAN needs no k and marks outliers as noise.
8	ANN (Backpropagation)	MLP: 135 inputs → 64 ReLU hidden → 20 softmax outputs, Adam optimiser, converged in 140 epochs; accuracy 0.99. Loss falls steadily (see plot).

6.1 Result Plots (ROC, Confusion Matrix, Clusters, ANN Loss)



7. Evaluation Metrics — Formulas to Remember

Metric	Formula / meaning
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$ — of predicted positives, how many are correct
Recall (Sensitivity, TPR)	$TP / (TP + FN)$ — of actual positives, how many were found
F1-score	$2 \cdot P \cdot R / (P + R)$ — harmonic mean
Specificity	$TN / (TN + FP)$; $FPR = 1 - \text{Specificity}$
ROC curve / AUC	TPR vs FPR at all thresholds; AUC = 1 perfect, 0.5 random
DET curve	FNR vs FPR on normal-deviate axes; used in speaker/biometric systems
Macro average	Average of per-class metric (each class equal weight) — used here for 20 classes
Silhouette score	$(b - a) / \max(a, b)$; -1 to 1, higher = better separated clusters
ARI	Agreement between clusters and true labels; 1 = perfect, ~0 = random

8. Important Code (ready to use in practical)

8.1 Imports

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MultiLabelBinarizer, OneHotEncoder, label_binarize
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import BernoulliNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_auc_score, roc_curve)
from sklearn.cluster import AgglomerativeClustering, DBSCAN
from sklearn.metrics import silhouette_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

8.2 Load, EDA & Preprocessing

```
df = pd.read_csv("datasets/candidate_job_role_dataset.csv")
print(df.shape); print(df.info()); print(df.isnull().sum())
print(df["job_role"].value_counts())
df["experience_level"] = df["experience_level"].str.strip().str.rstrip(",")
df = df[df.groupby("job_role")["job_role"].transform("count") >= 5]

skills = df["skills"].apply(lambda s: [x.strip() for x in s.split(",")])
mlb = MultiLabelBinarizer(); Xs = mlb.fit_transform(skills) # multi-hot
ohe = OneHotEncoder(sparse_output=False)
Xc = ohe.fit_transform(df[["qualification", "experience_level"]]) # one-hot
X = np.hstack([Xs, Xc]); y = df["job_role"].values
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2,
                                       stratify=y, random_state=42)
```

8.3 Train & Evaluate Classifiers (NB, ID3, KNN, SVM, RF, ANN)

```
models = {
    "NaiveBayes": BernoulliNB(),
    "ID3": DecisionTreeClassifier(criterion="entropy", random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "LogReg": LogisticRegression(max_iter=2000),
    "SVM": SVC(probability=True),
    "RF": RandomForestClassifier(n_estimators=200, random_state=42),
    "ANN": MLPClassifier(hidden_layer_sizes=(64,), max_iter=1000),
}
for name, m in models.items():
    m.fit(Xtr, ytr); pred = m.predict(Xte)
    print(name, "Accuracy:", accuracy_score(yte, pred))
    print(classification_report(yte, models[name].predict(Xte)))
```

8.4 ROC-AUC (multi-class, One-vs-Rest)

```
classes = np.unique(y); Yte = label_binarize(yte, classes=classes)
prob = models["NaiveBayes"].predict_proba(Xte)
print("Macro AUC:", roc_auc_score(Yte, prob, average="macro", multi_class="ovr"))
fpr, tpr, _ = roc_curve(Yte.ravel(), prob.ravel())
plt.plot(fpr, tpr); plt.plot([0, 1], [0, 1], "--")
plt.xlabel("FPR"); plt.ylabel("TPR"); plt.title("ROC"); plt.show()
```

8.5 Clustering

```
agg = AgglomerativeClustering(n_clusters=20, linkage="ward").fit(X)
print("Silhouette:", silhouette_score(X, agg.labels_))
db = DBSCAN(eps=1.5, min_samples=5).fit(X)
print("DBSCAN clusters:", len(set(db.labels_)) - (1 if -1 in db.labels_ else 0))
```

8.6 FIND-S (from scratch)

```
def find_s(examples):
    h = None
    for x, label in examples:
        if label == "Yes":
            # only positive examples are used
            h = list(x) if h is None else [a if a == b else "?" for a, b in zip(h, x)]
    return h
```

8.7 Project core — Answer relevance (TF-IDF + cosine) — evaluator.py

```
vectorizer = TfidfVectorizer(stop_words="english", ngram_range=(1, 2))
tfidf = vectorizer.fit_transform([question, answer])
sim = cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0] # 0..1
score += sim * 35 # relevance points
```

8.8 Project core — Weighted final score & grade

```
WEIGHTS = {"answer_relevance": 0.35, "grammar": 0.20, "vocabulary": 0.15,
           "confidence": 0.15, "communication": 0.15}
total = (relevance * 0.35 + grammar * 0.20 + vocab * 0.15
        + confidence * 0.15 + communication * 0.15)
# vocabulary: Type-Token Ratio
words = re.findall(r"\b[a-zA-Z]{3,}\b", text.lower())
ttr = len(set(words)) / len(words)
```

8.9 Project core — ATS role match (role_matcher.py)

```
match_score = round((skills_score * 0.40 + content_score * 0.20
                    + exp_score * 0.20 + qual_score * 0.20))
if content_score < 40 or word_count < 60: # not a real resume
    match_score = min(match_score, 25)
```

9. Likely Viva Questions — Short Answers

Q. What type of ML problem is your project?

Supervised multi-class classification (predict job role from skills, qualification, experience), plus NLP similarity scoring and a weighted linear scoring model. Clustering is used for unsupervised grouping.

Q. What are your features and target?

Features: 116 multi-hot skill columns + one-hot qualification and experience (135 total). Target: job_role.

Q. Why multi-hot encoding for skills?

A candidate has several skills at once, so each skill becomes its own 0/1 column. Label encoding would wrongly imply an order between skills.

Q. Why BernoulliNB and not GaussianNB?

All features are binary (present/absent). Bernoulli NB models binary features; Gaussian assumes continuous, normally distributed values.

Q. Which model is best?

On the random split all score ~99%. On the leakage-free GroupKFold evaluation Random Forest is best (~92%), because averaging many trees reduces variance.

Q. What is data leakage here?

The same duplicate row appears in both training and test sets, so the model is tested on data it has already seen, inflating accuracy.

Q. Where is TF-IDF used and why?

To compare the spoken answer with the question and the ideal answer. TF-IDF down-weights common words and highlights topic words; cosine similarity measures the angle between the two vectors, independent of answer length.

Q. What is entropy / information gain in ID3?

Entropy measures impurity: $H = -\sum p \log_2 p$. ID3 splits on the attribute with the highest information gain (largest reduction in entropy).

Q. Difference between FIND-S and Candidate-Elimination?

FIND-S gives only the single most specific hypothesis and ignores negatives. Candidate-Elimination uses positives and negatives and outputs the whole version space (S and G boundaries).

Q. Why is the dataset linearly separable?

Each role has a distinct set of skills, so a hyperplane can separate every role in the 135-dimensional one-hot space; a hard-margin linear SVM reached 100% training accuracy.

Q. How does backpropagation work?

Forward pass computes outputs, loss (cross-entropy) is calculated, gradients are propagated backwards with the chain rule, and weights are updated by gradient descent (Adam) — repeated each epoch until the loss converges.

Q. Agglomerative vs DBSCAN?

Agglomerative is bottom-up hierarchical merging (needs number of clusters, gives a dendrogram). DBSCAN is density-based, finds arbitrary-shaped clusters, needs eps and min_samples, and labels outliers as noise.

Q. Is there regression in your project?

The final score and ATS score are continuous and computed as a weighted linear sum — the same form as linear regression, but with rubric weights. Learning these weights from human-rated answers with Linear Regression is future work.

Q. How is confidence measured from audio?

Audio split into 0.5 s chunks; loudness (dBFS) variance gives volume consistency, silent-chunk ratio gives pause ratio, plus the speech-to-text confidence.

Q. Why ROC-AUC with macro average?

It is threshold-independent and, averaged One-vs-Rest over all 20 classes equally, it is not dominated by any single class.

10. Limitations & Future Work

- Dataset is small and heavily duplicated (98 unique profiles) → collect more diverse real resumes.
- Scoring weights are hand-set; learn them from human-rated interviews (Linear Regression / Gradient Boosting).
- TF-IDF matches words, not meaning → use sentence embeddings (e.g. BERT/SBERT) for semantic relevance.
- Speech-to-text errors affect every downstream score; accents and background noise reduce accuracy.
- Add cross-validation and hyper-parameter tuning (GridSearchCV) for the classifiers.

11. Conclusion

The project combines NLP (resume parsing, TF-IDF similarity), classical supervised classifiers (Naive Bayes, ID3, KNN, SVM, Random Forest, ANN), unsupervised clustering and audio feature engineering into one end-to-end interview-practice system. Experiments on the candidate-role dataset show the classes are linearly separable and all models perform very well on a random split, while a leakage-free grouped evaluation shows Random Forest generalises best (~92%).